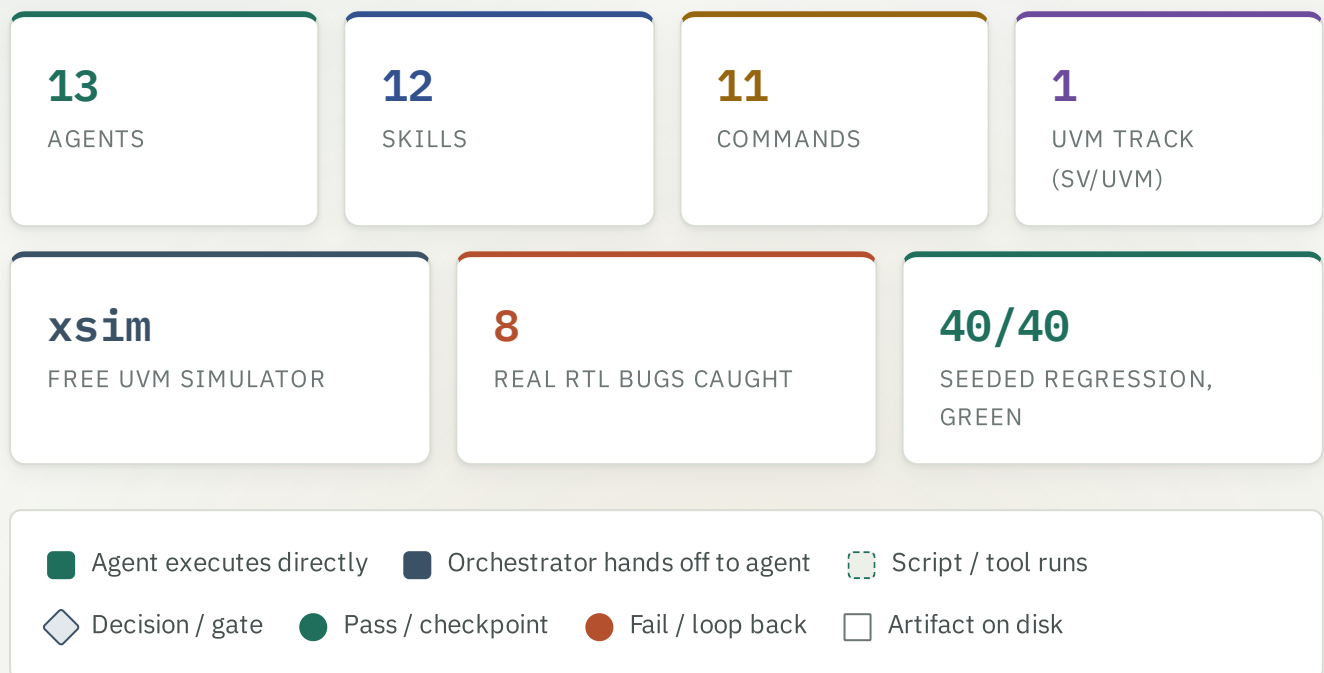


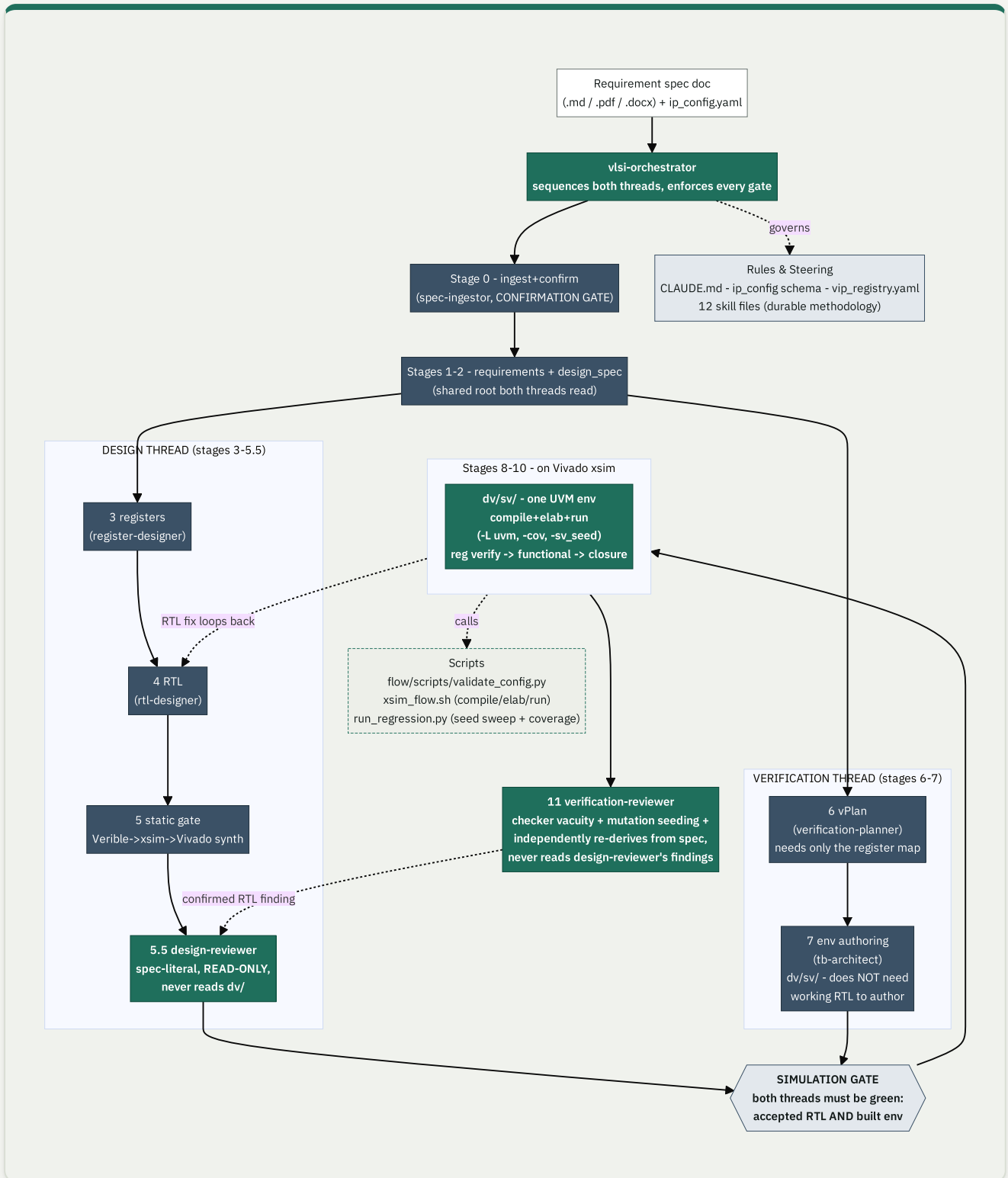
VLSI Agent Pipeline

How the front-end VLSI agentic framework carries an IP from a requirement document to coverage closure — every stage, every gate, every rule that governs it — with one verification methodology, standard SystemVerilog UVM, run on AMD Vivado xsim. Illustrated against the reference IP: PMTPC-4, a 4-channel timer/PWM controller.



1 · System architecture / 2 · Ingest & confirm / 3 · Spec /
 4 · Design thread — registers, RTL & static gate / 5 · Verification thread — vPlan & UVM env /
 5a · How a cycle gets checked / 6 · Test authoring / 7 · Coverage closure loop /
 8 · Independent review / 9 · Signoff / 10 · Lessons learned / 11 · Key results /
 12 · Human vs. agent effort / 13 · Future improvements

One orchestrator, thirteen specialists, two independent threads that converge only at the simulation gate — each with its own review agent that never reads the other thread's collateral.

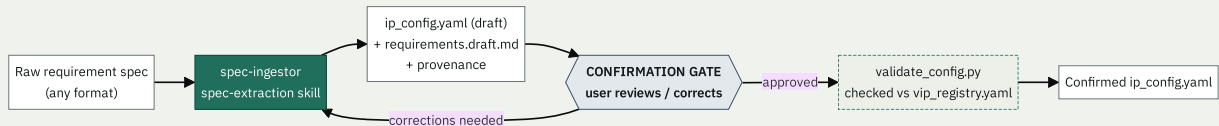


The design thread and verification thread run independently from the moment requirements.md/design_spec.md exist — vPlan authoring only needs the register map, and env authoring (RAL, scoreboard skeleton, coverage model, SVA) doesn't need working RTL, only RTL's *interface*. The two threads share nothing else until the simulation gate, where verification's stimulus runs against design's RTL for the first time. Each thread carries its own independent review agent, scoped to its own artifacts and explicitly forbidden from reading the other thread's collateral — design-reviewer never opens dv/, verification-reviewer re-

derives expected checker behavior from spec before ever comparing it to the RTL, and neither reads the other's report before finishing its own.

02 Stage 0 — Ingest & Confirm

Agent: spec-ingestor · The front door. Nothing downstream runs until a human approves.



CLAUDE.md Generic-first: nothing hardcodes an IP's details — everything reads from `ip_config.yaml` .

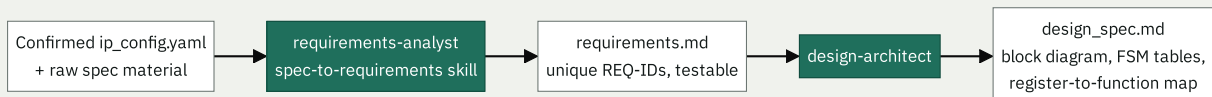
orchestrator.md "The ingestion confirmation gate is mandatory." Never run design/RTL/verification on an unconfirmed, auto-extracted config.

spec-ingestor.md Every extracted assumption is tagged in `provenance.needs_confirmation` — surfaced, never buried.

✓ Confirmed config validates against schema + VIP registry

03 Stage 1-2 — Requirements & Design Spec

Agents: requirements-analyst, design-architect · Builds the root of the traceability spine.



spec-to-requirements

Every requirement gets a stable, atomic, testable ID — never reused or silently renumbered.

CLAUDE.md

Traceability chain begins here:

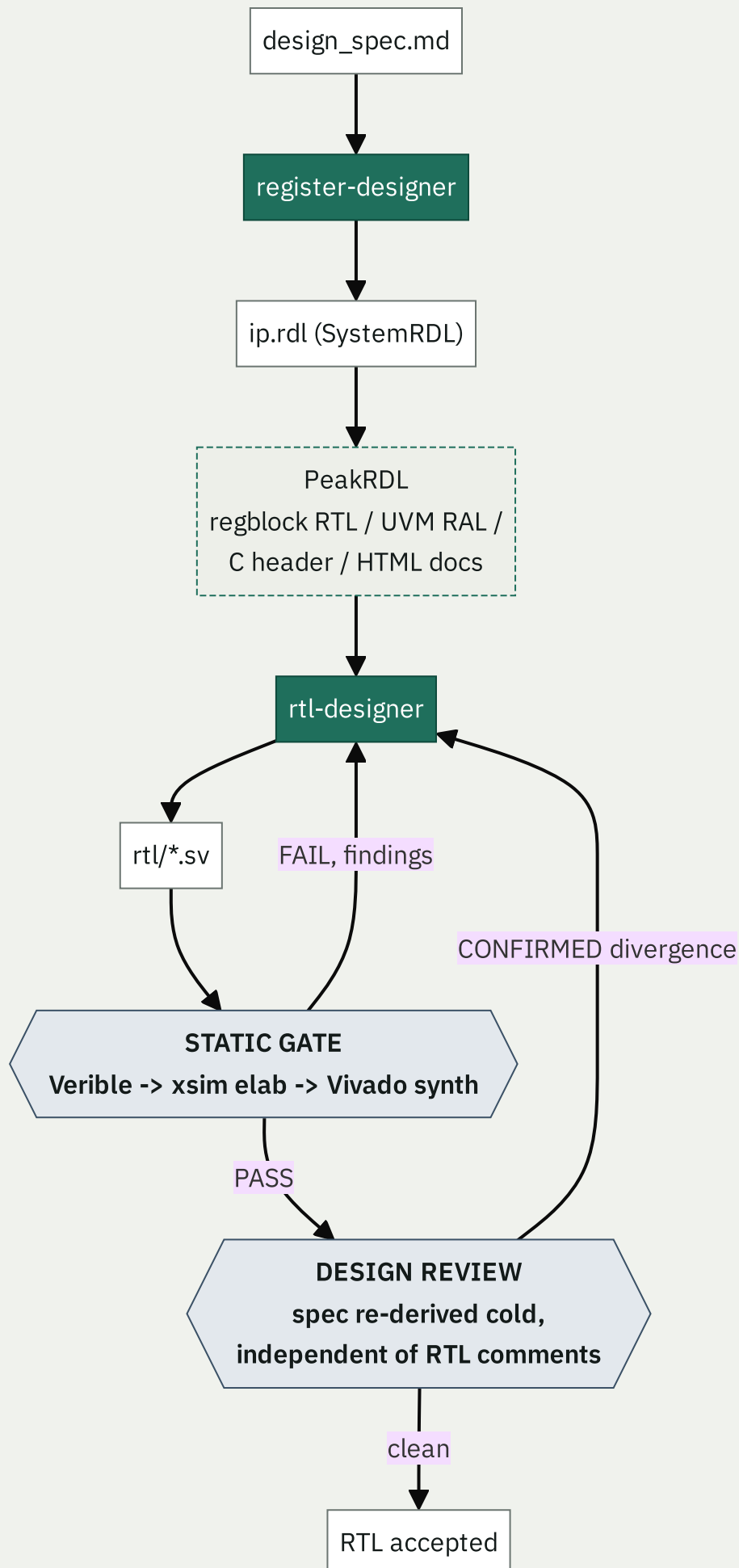
REQ → SPEC → REG → RTL → VPLAN → COV .

Real example — PMTPC-4: 22 requirements across 6 categories (core timer/PWM, interrupts, APB interface, reset/control, non-functional) — each traced all the way to a covergroup bin in the final closure report.

04

Design Thread (Stages 3-5.5) — Registers, RTL & the Static / Design Gates

Agents: register-designer, rtl-designer, lint-static-checker, design-reviewer · Runs independently of the verification thread below — the only thing they share is the confirmed spec they both started from. Two independent gates within this thread, not one.



lint-static-checker.md

One command,

flow/scripts/static_gate.py : Verible lint, xsim elaboration of the DUT alone, Vivado out-of-context synthesis. Deny-by-default — any unclassified warning fails; per-IP waivers need a written reason.

rtl-designer.md

Use

{default: '0} , not a bare

'0 , on an unpacked

hwif_in /

hwif_out struct — unambiguous for every tool.

rtl-designer.md

No tool pragmas in RTL comments; keep operand widths equal in expressions (the static gate does not flag a width mismatch inside an expression).

design-review skill

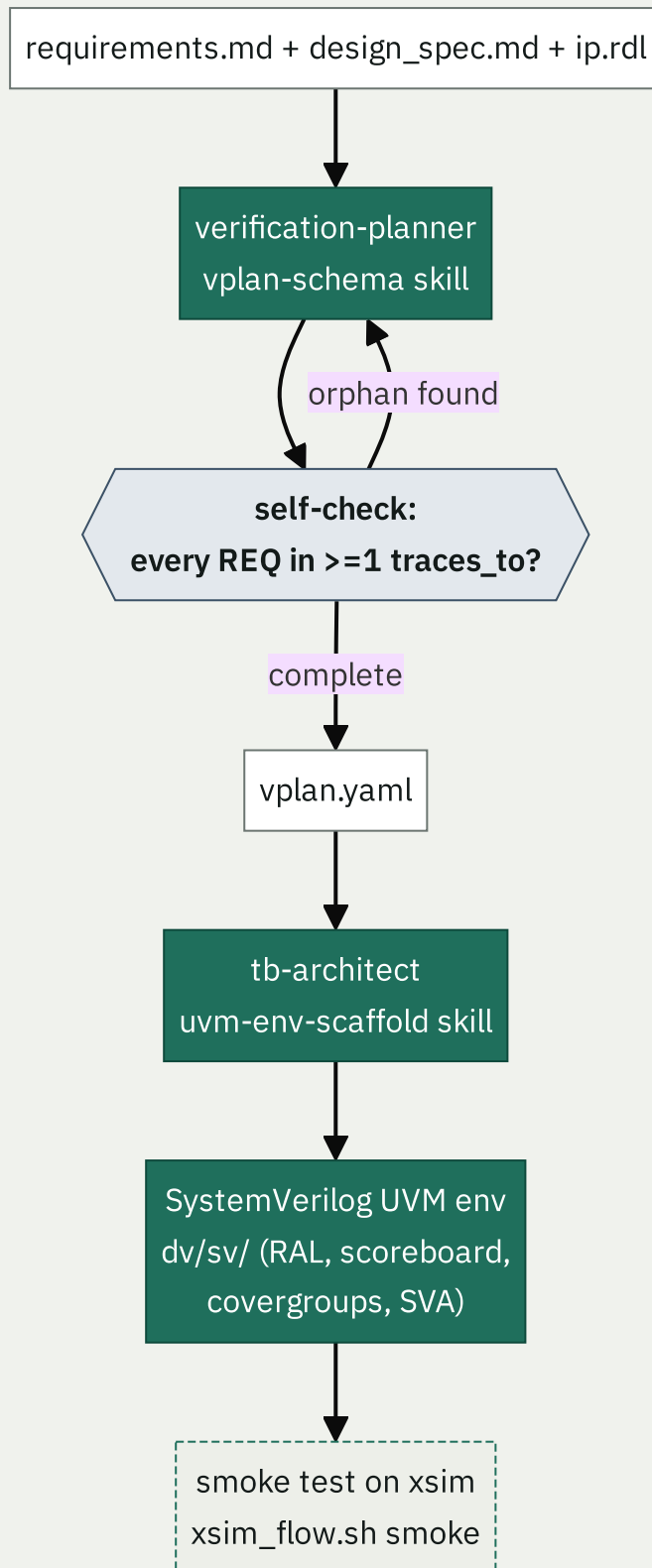
"Takes effect at the next X, not immediately" in a spec implies a shadow register latched at that event — check

every field described that way, not just the first one implemented.

Real example — PMTPC-4: four RTL bugs shipped past the static gate before being caught: a protocol phase decoded from a registered instead of combinational state (PREADY one cycle late on every transfer), a skipped FSM state on periodic reload, a missing hardware-clear on SOFT_RESET, and a mid-RUNNING COMPARE write applying live instead of shadowed. The design review also cross-referenced the Register Spec against the Design Spec's own FSM table and confirmed a fifth, initially-suspected gap (CH_EN=0 during LOAD/EXPIRED) was actually already correct — the Register Spec's field description said "regardless of current state," which the Design Spec's transition table alone didn't state.

✓ Static gate PASS (Verible + xsim + Vivado synth), and design review finds no confirmed divergence

Agents: verification-planner, tb-architect · Runs in parallel with the design thread above — vPlan needs only the register map, and env authoring doesn't need working RTL. One SystemVerilog UVM env, built at full capability and actually run on xsim once both threads reach the simulation gate.



vplan-schema

No orphan vPlan items, no requirement left untraced — checked here, before RTL/env effort is spent, not just flagged later at closure.

uvm-env-scaffold

One env, generated at full closure-grade capability, with the xsim compatibility rules baked in (filelist

-i form; RAL block aliased +
new() ; white-box SVA bound to registered signals).

tb-architect.md

Bring up the smoke test (reset + one RAL access) and confirm it compiles, elaborates, and runs clean on xsim before elaborating the full suite.

Real example — PMTPC-4: the SV/UVM env compiles and elaborates on xsim via -L uvm (built-in UVM 1.2) and runs the sanity + error tests green (UVM_ERROR 0, scoreboard checking, 0 SVA failures). Bringing it up on a real UVM simulator immediately caught four bugs the previous Python-only flow never could — a reserved-keyword task name, a RAL/DUT class-name collision, a non-factory RAL block, and a white-box-SVA X-sampling issue.

05a

How a Cycle Gets Checked

Standard SystemVerilog UVM on xsim — the components, and how they combine to check the DUT every clock.

This is a conventional UVM environment: a reusable protocol UVC, a layered env with RAL and a reference-model scoreboard, functional coverage sampled by the monitor, and a bound SVA layer — all compiled, elaborated, and run on Vivado xsim with its built-in UVM 1.2 (-L uvm). Native rand/constraint/dist drive the stimulus; -sv_seed makes each seed reproducible.

The components

UVM COMPONENT	ROLE	WHERE
Driver / Sequencer	Reusable APB UVC drives the bus from constrained-random seq_items; the RAL runs through its reg adapter	vip/apb/sv/
Monitor	Samples the APB transactions and DUT outputs, publishes them on an analysis port every transfer	apb_monitor.sv

UVM COMPONENT	ROLE	WHERE
Reference-model scoreboard	Predicts register readback + protocol (PSLVERR, wait states) and checks the DUT against it	<code>pmtpc4_scoreboard.sv</code>
Functional covergroups	Register/config + white-box FSM covergroups, sampled by the monitor/scoreboard from a signal it just checked	<code>pmtpc4_coverage.sv</code> / <code>sva/</code>
Code coverage (stmt/branch/cond/toggle)	xsim's own RTL instrumentation via <code>xelab -cc_type sbct</code> , merged across seeds with <code>xcrg</code>	<code>xelab --cov</code>
SVA protocol assertions	Bound white-box modules asserting APB handshake, PWM waveform, IRQ aggregation, freeze, reset	<code>dv/sv/sva/*.sv</code>
RAL	PeakRDL-generated <code>uvm_reg_block</code> wired via the adapter + an explicit predictor (auto-predict off)	<code>pmtpc4_ral_pkg.sv</code>

Step by step: how a single cycle gets checked

- 1 A UVM sequence randomizes a `seq_item` (native `rand/constraint`) and sends it to the driver — e.g. a write to `CH0_COMPARE`.
- 2 The driver drives the APB bus; the monitor samples the transaction and DUT outputs (`prdata`, `pready`, `pslverr`, `pwm_out`, `irq`, `INT_STATUS`) and publishes them.
- 3 The scoreboard/RAL predictor independently computes what those signals *should* be from the spec/RAL model — not from the RTL — and compares, signal by signal.
- 4 The bound SVA layer checks cycle-accurate relationships continuously (handshake timing, waveform rule, freeze, valid-state) in parallel with the transaction checks.
- 5 **Coverage is sampled only from a signal that was just checked** — by the monitor/scoreboard, never by a test calling `.sample()`. Coverage is a side effect of a passing check, not an independent claim.

`run_regression.py` repeats this across every test × N seeds (`-sv_seed`), collecting



6

functional + code coverage per run.

7

`coverage-closure` merges the databases with `xcrg`, triages every unhit bin (add stimulus, waive with a structural reason, or file as a bug), and loops to the goal.

8

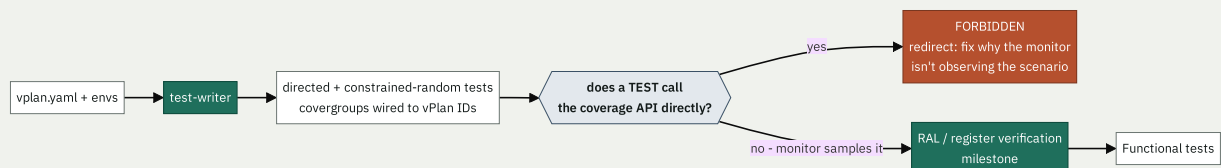
`verification-reviewer` then **deliberately seeds a bug** (e.g. a flipped comparison), reruns, and confirms the checks actually fail — proof the scoreboard/SVA isn't decorative — then reverts.

xsim gotcha, from the real bring-up: a white-box channel assertion fired every clock even though `state_o` was correct — xsim samples the *preponed* value of a continuous-assign net (`assign state_o = state;`) as X. The fix is to bind the SVA to the registered state reg, not its combinational alias. This class of simulator-specific lesson is exactly what a real UVM sim surfaces and a generated-but-never-run env hides.

06

Stage 8-9 — Test Authoring

Agent: test-writer · The single most important rule in the whole pipeline lives here.



test-writer.md

Coverage may only be sampled by the monitor/scoreboard, from a signal it just checked — never by test/sequence code calling

`.sample()`

directly. A test's job is to

`create`

a scenario, never to declare it covered.

test-writer.md

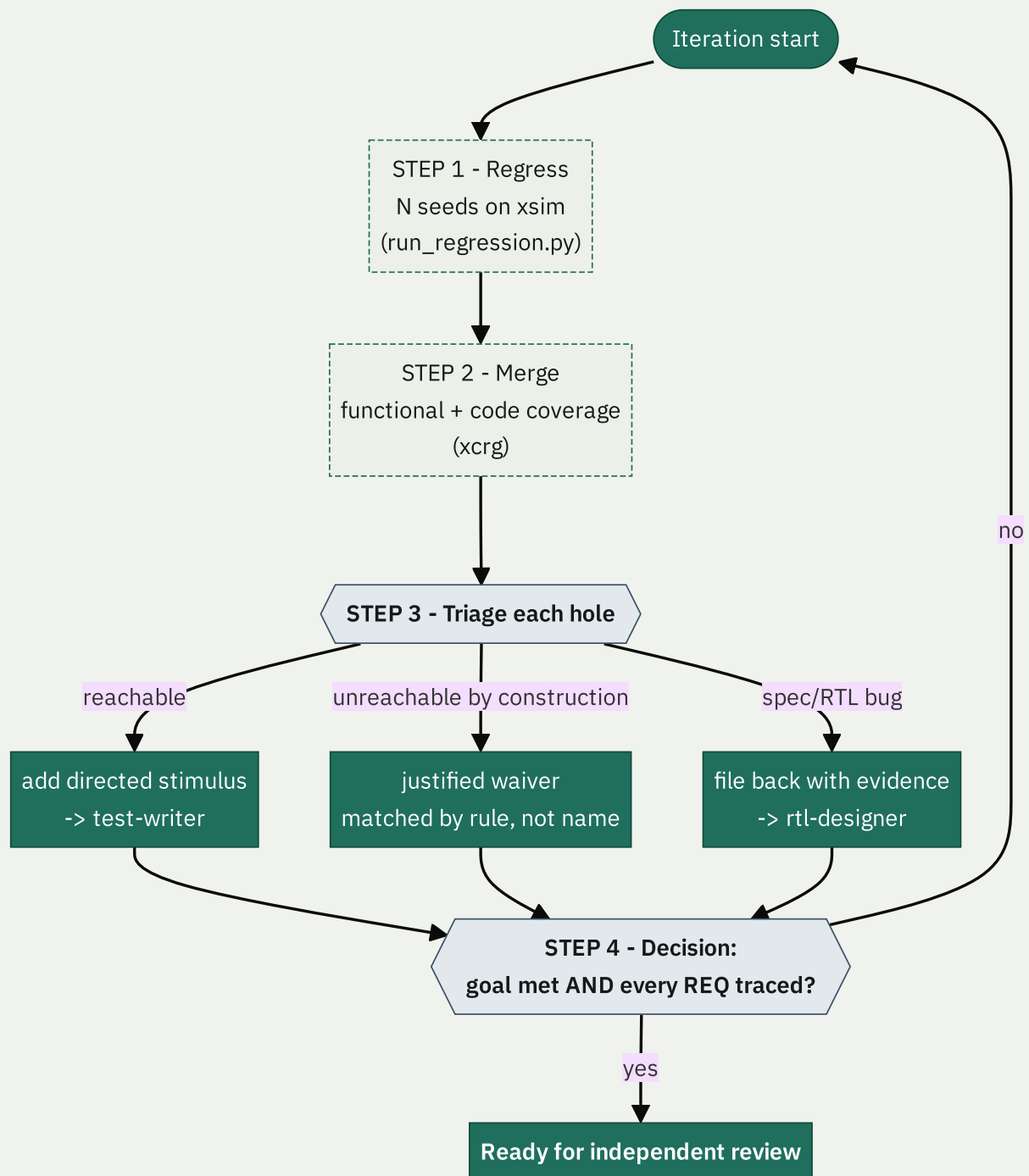
Every test maps to ≥ 1 vPlan ID; self-checking against the scoreboard/RAL, never eyeballing waveforms.

Hard-won rule: an environment can reach "100%" functional coverage while checking almost nothing — if tests call the coverage API directly to mark bins covered, disconnected from any signal actually observed. Coverage must be sampled only by the monitor/scoreboard, from a signal it just

checked. This is why `test-writer.md` states the rule explicitly instead of leaving it implied — the single most important discipline in the pipeline.

07 Stage 10 — Coverage Closure Loop

Agent: coverage-closure · Regress, merge, triage, iterate — until the number is real.



coverage-triage Classify every hole: reachable → stimulus, unreachable → justified waiver, bug → file back with evidence. Never silently drop one.

coverage-closure.md "100% functional coverage with failing/absent checks is not closure" — pass/fail and coverage must both be green, and the scoreboard must actually be checking.

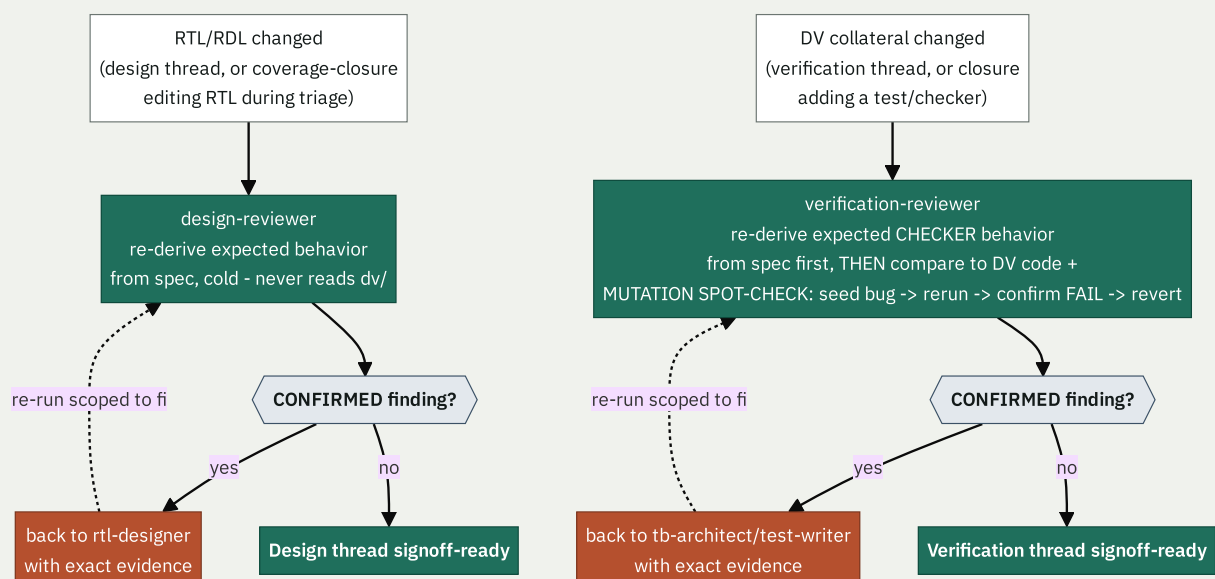
verification-review skill Waivers matched by rule/pattern, never a hand-maintained allow-list of point names — a pattern keeps catching new holes; a name list silently stops.

On PMTPC-4: the loop runs on xsim via `run_regression.py` (every test × N seeds) with xcrg-merged functional + code coverage. Four real RTL design bugs were found and fixed while verifying this IP (PREADY timing, skipped LOAD state, missing SOFT_RESET clear, live-vs-shadow COMPARE) — closure is about finding those, not just filling bins. xsim does not report *assertion* coverage, so assertion evidence rolls up as pass/fail plus SVA cover.

08

Independent Review — One Per Thread, Keyed to Its Own Changes

Agents: design-reviewer (design thread, stage 5.5), verification-reviewer (verification thread, stage 11) · Fresh context each run, no stake in the changes just made, and neither reads the other's report first.



design-reviewer.md Never reads

ips/<ip>/dv/ — independence from the verification thread is a stated rule, not an informal convention. Rebuild the contract from the spec

before reading the RTL; an inline comment's justification is a claim, never proof.

verification-reviewer.md Reads

requirements.md /

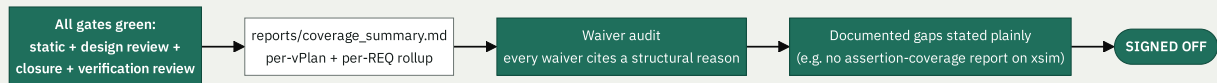
design_spec.md as a primary input, on equal footing with the DV collateral — independently re-derives what a checker

should check before looking at what it actually does, the same discipline design-reviewer applies to RTL. A checker comment that cites RTL behavior instead of a spec clause is itself a finding.

verification-review skill Mutation/bug-seeding is the standard way to prove a checker works: seed an obviously-wrong change, confirm something fails, always revert immediately — never leave a seeded bug in the tree.

both agents Report, don't fix, and don't read each other's findings before finishing your own — reviewing-and-fixing in the same pass, or anchoring on the other reviewer's conclusion, both re-introduce the confirmation bias independent review exists to avoid.

Real example — PMTPC-4: the verification-reviewer seeded two deliberate bugs — a comparison operator flip in the PWM channel core, and a removed write-gate in the APB error decode — in two different modules. Both were caught immediately by the existing checks (one via 6 reference-model errors, the other at three independent layers simultaneously). Both were reverted and confirmed clean via `git diff` before the full suite was re-run green. In a later pass, `design-reviewer` and `verification-reviewer` were dispatched at the same time against the same open question (a PWM-gating formula that looked narrower in the spec than in the RTL) — neither was shown the other's conclusion, and both independently recommended widening the spec to match the RTL rather than narrowing the RTL, for the same underlying reason (the spec's own stated requirement carried no state qualifier). Two independently-reached conclusions agreeing is real corroborating evidence in a way one reviewer's opinion never is.



Sign-off criteria (all required together):

100% functional coverage, net of justified waivers

100% code coverage, net of justified waivers

every requirement traced to a covered item

all seeds green: UVM_ERROR 0, 0 SVA fails, scoreboard checking

10 Lessons Learned

Nine discoveries from PMTPC-4's first run through the pipeline, plus a second, deeper audit pass (design thread and verification thread run as fully independent, mutually-blind reviews — see Section 8) that found five more real bugs and three durable methodology lessons. Full evidence for the second pass lives in `ips/pmtpc4/reports/coverage_waivers.md`; this table is the condensed, permanent-change version.

#	DISCOVERY	IMPACT	CHANGE MADE
1	Nested quoting through <code>ws1 bash -lc</code> silently corrupts <code>\$(...)</code> substitutions	Commands appeared to run but used empty/wrong values, with no error	Convention: write multi-step commands to a script file, never inline
2	Verilator (even latest) can't run complete UVM — coverage collection + full-	SV/UVM could be authored but never actually simulated	Adopted AMD Vivado xsim (free ML Standard) as the UVM simulator; Verilator demoted to a lint helper, later removed entirely (Vivado-only static gate)

#	DISCOVERY	IMPACT	CHANGE MADE
	testbench sim are incomplete		
3	hwif_in = '0; on an unpacked struct compiles under Yosys/sv2v, fails Verilator's C++ codegen	Static gate passed; simulation build failed	Use '{default: '0}'; documented in rtl-designer.md
4	A comment starting with the word "verilator" is parsed as a pragma	BADVLTPRAGMA compile error from plain prose	Documented as an explicit style rule
5	First-generation coverage model sampled bins directly from test code	"100%" coverage that checked almost nothing	Rebuilt around a cycle-accurate reference model; anti-pattern rule added to test-writer.md
6	peakrdl-regblock's field_combo struct trips Verilator MULTIDRIVEN on every IP	Would be rediscovered per-IP	Shared, filename-matched flow/tools/verilator_waivers.vlt
7	On xsim, a white-box SVA bound to a combinational output alias samples X in the preponed region every clock	Valid-state assertion fired spuriously on every cycle despite correct RTL	Bind white-box SVA to the registered signal, not the assign alias; documented in uvm-env-scaffold
8	Mid-RUNNING COMPARE write applied live instead of shadowed (Design Spec §7.4)	Passed lint, elaboration, and first-pass sim	Caught only by literal spec re-read; shadow register added, model fixed to match
9	No independent review stage existed	Coverage-closure graded its own homework	design-reviewer / verification-reviewer added, validated day-one via 2 live mutation spot-checks
10	PWM stayed high one extra cycle after soft-disable (F1) — a checker/coverage bin had marked the correct behavior "structurally unreachable," hiding the bug from both the	Passed the entire existing regression; only found by a fresh, cold RTL-vs-spec re-derivation	RTL fixed; downgraded the covergroup bin, added a dedicated property as the real guard (a_pwm_low_in_idle)

#	DISCOVERY	IMPACT	CHANGE MADE
	coverage report and a casual read		
11	A field with both hwset and a coincident SW write defaulted to precedence=sw in SystemRDL, silently dropping a one-shot self-clear (F3)	Race only manifests when a write lands the exact cycle the hardware self-clears — invisible to typical directed stimulus	precedence=hw added to the RDL field; closed with a "spray of writes" test across the whole race window rather than trying to hit one exact cycle
12	A freeze (MODULE_EN=0) silently swallowed a CH_EN/CH_START transition that arrived while frozen (F4)	The natural bring-up order (disable, configure, enable, then enable module) left every channel stuck in IDLE forever	Latched the pending transition through the freeze window; fixing it also required updating two independent checker mirrors that had gone stale relative to the new RTL — exactly the failure mode spec-independence discipline exists to catch
13	An assertion cross (cross A, B;) with no iff of its own samples every clock, not gated by its coverpoints' iff — a genuine SystemVerilog semantics gap, not a typo (F15)	One cross's true sample count was ~560x its own coverpoint's correctly-gated count; "100% covered" meant nothing about the intended event	Found in 5 crosses across one environment; rule added to tb-architect.md to always give a cross the same iff as its coverpoints
14	"Delete a named alias wire, inline its expression at the one use site" looks like the same fix that worked once (a mixed-bits signal split into two <i>other named</i> signals) but is a different move with the opposite effect — confirmed by direct measurement, not theory	One module's toggle-coverage score dropped from 100% to 40% after "fixing" it this way; an anonymous inlined expression has no name left for the waiver file to reference	Reverted in full; explicit anti-pattern warning added to coverage-triage distinguishing the two techniques
15	A hierarchical port reference between two module instances passes Verible + Verilator cleanly, but	Would have been a synthesis-breaking bug (32-bit write data truncated to 1	Reverted; rule added to lint-static-checker.md to treat any Yosys "Resizing cell port" warning as a hard gate failure

#	DISCOVERY	IMPACT	CHANGE MADE
	sv2v silently truncates its width when flattening for Yosys	bit) had the full 3-tool gate not been run before trusting the change	

11 Key Results – PMTPC-4

4-channel timer/PWM controller, APB, 5 binding errata. The reference IP for the single-track SV/UVM + xsim framework. Numbers below are from the live regression as of the most recent run, not aspirational.

METRIC	VALUE
Verification methodology	standard SystemVerilog UVM (single track)
Simulator	AMD Vivado xsim (free ML Standard)
SV/UVM env on xsim	compiles + elaborates + runs (-L uvm)
Seeded regression	40/40 runs passed – UVM_ERROR 0, 0 SVA fails, scoreboard checking
Constrained randomization	native rand/constraint/dist, -sv_seed
Functional coverage (union)	99.7% – sole residual is one signed-off waiver (cg_apb.cp_wait.many)
Code coverage – statement / branch / condition	99.9% / 98.1% / 100.0%
Code coverage – toggle	DUT code toggle 458/490 bits = 93.5% (100% net of 1 documented constant net), measured on a separate toggle snapshot (no binds, no \$dumpvars) proven identical to the normal run; register block measured separately as register-bit toggle 486/486 = 100% (reg_bit_toggle_cov). The earlier 46.0% was a measurement artifact – see coverage_waivers.md, 2026-09-26 correction
RTL design bugs found & fixed (first pass + second audit pass)	4 + 4 (F1/F3/F4/F5)

METRIC	VALUE
xsim bring-up bugs caught & fixed	4 (keyword, RAL collision, RAL new, SVA X-sampling)
Coverage-model correctness bugs found (second audit pass)	1 class, 5 instances (F15 – ungated covergroup crosses)
Toolchain gotchas caught by the static gate before shipping	1 (sv2v silently truncating a hierarchical port reference's width)
Assertion coverage	not reported by xsim – evidence via scoreboard + SVA cover
Source	this repository (ips/pmtpc4/, tag pmtpc4-v1.0)

12 Human vs. Agent-Assisted Effort

Illustrative from this one real run – not a large-sample study, but every line below happened.

STAGE	WHAT THE AGENT DID ALONE	WHAT NEEDED THE HUMAN
Environment setup	Researched the simulator options, wrote the xsim flow-runner scripts, diagnosed the xsim compile/elab/run gotchas	Installed Vivado xsim (AMD account + ~50 GB download) and ran the sudo apt lines – no interactive channel reaches the agent
Simulator strategy	Compared Verilator / Questa Starter / xsim against the constraints, proved the SV/UVM env runs on xsim end-to-end	Set the constraints (open-source-or-free, local+headless, standard UVM) and approved the dual-track purge – a scope call
Coverage closure	Ran the full 25-seed regression, triaged every hole, rebuilt the coverage model, fixed 3 RTL bugs autonomously	Reviewing and accepting those 3 fixes as correct design changes
Spec ambiguity (COMPARE shadow)	Re-derived the correct behavior from the spec, implemented the fix, added a targeted test, re-proved closure	Flagged the ambiguity for the user's attention before it was resolved
Independent review	Built the review methodology from real signoff practice (OpenTitan checklist,	Chose to create two review agents (design + verification) rather than

STAGE	WHAT THE AGENT DID ALONE	WHAT NEEDED THE HUMAN
	mutation-testing literature), ran 2 live bug-seeding spot-checks end-to-end	one, on the user's explicit direction

12a What Only Humans / Only Agents Can Do

ONLY HUMANS	
Grant sudo / interactive credentials	No interactive password channel reaches the agent
Set simulator constraints & approve the single-track direction	Cost/scope tradeoff, not a technical fact
Accept an agent-authored RTL bug fix as correct	Design accountability
Decide review-agent scope (1 vs. 2 agents)	Architectural judgment call, explicitly delegated here

ONLY THE AGENT (AT THIS SPEED)	
Cross-reference 3 spec documents per requirement, 22 times	Human working memory can't hold all three simultaneously
Run 25 full regressions + 2 live mutation experiments in one session	Would be days of human wall-clock, done in minutes of compute
Rebuild an entire coverage model and re-verify 245 bins stayed correct	Mechanical, exhaustive, and easy for a human to under-check under deadline
Diagnose an xsim preponed-sampling gotcha by printing \$sampled vs the settled value	Requires simulator-semantics reasoning, not domain knowledge

13 Future Improvements

Three of the original four P0 items are done — kept below, struck through, for the record — plus the real gaps this session's deeper audit surfaced, prioritized by what blocks the next IP.

PRI	ITEM	PROBLEM TODAY	PROPOSED SOLUTION
DONE	Finish pmtpc4 closure on xsim	41/41 seeded regression green, 99.7% functional, DU code toggle 93.5% (100% net of 1 documented constant), register-bit toggle 100% — see Section 11.	
DONE	Coverage report roll up to REQ level	flow/scripts/coverage_report.py generates coverage_summary.md (per-vPlan + per-REQ) straight from vplan.yaml + regression.txt + xcrg output — nothing hand-edited.	
DONE	Generic templated engine unbuilt	flow/templates/full_test.sv.j2 and tb_top.sv.j2 now exist and are used by the flow.	
DONE	Toggle coverage residual	Root cause was the testbench, not the tool: a \$dumpvars or a bound checker anywhere in the xsim design silently stops toggle recording on the nets they touch. Fixed generically: separate toggle snapshot (xsim_flow.sh --toggle), binds/dump moved to their own top modules, register block measured through the RAL (reg_bit_toggle_cov), bit-weighted reports/_cov/toggle_summary.txt.	
P1	Deferred RTL-split follow-up (cpuif_wr_data, int_status_val/int_en_val, ch_expiry[3])	Investigated this session; every attempt (hierarchical port reference, inline-and-delete-the-wire) made the score worse or broke the static gate — see Section 10, rows 14-15	Leave as documented residual unless a genuinely new technique is found and verified with a real before/after regression, never assumed from the one precedent (cpuif_rd_data_pad) that did work
P1	Mutation spot-check is manual	The reviewer hand-picks a few lines per run — thorough but not systematic	A small script that seeds N random single-line RTL mutations, reruns the regression, and reports the catch rate
P2	UVM 2020 instead of built-in 1.2	xsim ships UVM 1.2; the bundled Accellera env/_tools/uvm-core sources aren't compiled in yet	Wire an opt-in verification.uvm_version "2020" path that compiles the bundled sources into xsim
DONE	Formal (SymbiYosys) drafted but never run	Removed: the flow is Vivado-only; the never-run SymbiYosys harness was deleted with Verilator, Yosys and sv2v.	

PRI	ITEM	PROBLEM TODAY	PROPOSED SOLUTION
P3	Cross-IP pattern library	Every peakrdl-regblock false-positive, xsim/UVM gotcha, and coverage-tool blind spot (this session added several — Section 10) was learned fresh on this one IP	A pattern library applied automatically to the next IP in the family

VLSI Front-End Agentic Framework — single-track SystemVerilog UVM on AMD Vivado xsim. Toolchain: PeakRDL, Verible (free/open-source) + Vivado (free ML Standard): xsim for simulation and coverage, xelab + synth_design for the static gate.